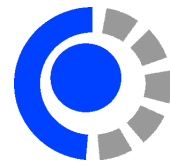




Sistemas Operativos I 2024 Recuperatorio Segundo parcial



Temario

- Procesos y threads. Comunicación entre procesos. Funciones POSIX para acceso al sistemas de archivos (abstracción open, read, write, close). Drivers de dispositivos de E/S.

Software y Hardware

El parcial se realiza sobre las computadoras de laboratorio. **Utilizar la versión de Linux de los laboratorios, y XINU. Sólo se puede utilizar la web de la materia, PEDCO, editor pluma y las herramientas de desarrollo. NO se puede utilizar servicios de google, drive, chat, etc.**

- Desarrollar dos programas en **LINUX** que trabajen cooperativamente para procesar este archivo:

<https://se.fi.uncoma.edu.ar/so/misc/desordenado.bin>

El primer programa lee el contenido de un archivo llamado desordenado.bin, en donde existen números enteros, y los coloca en un segmento de memoria compartida.

El segundo programa ordena los números.

Finalmente, el primer programa guarda los números en un nuevo archivo llamado ordenado.bin.

ATENCION: habrá que coordinar los dos programas.

Para esto, se utilizarán un número entero ubicado en el primer lugar de la memoria compartida:

- El primer programa, luego de crear la memoria compartida, coloca el entero cero (0) en el primer lugar de la memoria compartida. Luego de eso, colocará los números del archivo desordenado.bin en las posiciones siguientes. Cuando termina de cargar la memoria, coloca en el primer lugar de la memoria compartida el entero n, que sea la cantidad de enteros del archivo desordenado.bin
- El segundo programa, hará espera por consulta. Estará leyendo el primer lugar de la memoria compartida, esperando a que el valor sea diferente de cero. Cuando sea diferente de cero, significa que el primer programa habrá terminado de cargar la memoria. Entonces, el segundo programa puede comenzar en ese momento a ordenar los números. Finalmente, una vez ordenados, colocará el valor uno (1), en el primer lugar de la memoria compartida.
- El primer programa se quedará esperando de manera activa. Se quedará esperando a que en el primer lugar de la memoria compartida aparezca el uno (1). Cuando ese lugar sea 1, sabrá que el segundo programa finalizó, y podrá continuar y guardar los números enteros en el nuevo archivo ordenado.bin.

Para poder leer y escribir en la memoria compartida, hay que utilizar el flag O_RDWR en shm_open() y los flags PROT_READ|PROT_WRITE en mmap(). Por ejemplo, el programa que crea la memoria compartida hace algo como:

```
shm_fd= shm_open("PARCIAL", O_CREAT | O_RDWR , 0666);  
etc  
ftruncate(.....etc) etc  
int * ptr= mmap(0, [aquí va el tamaño], PROT_READ | PROT_WRITE, MAP_SHARED, shm_fd,0);
```

Y en el segundo programa algo así:

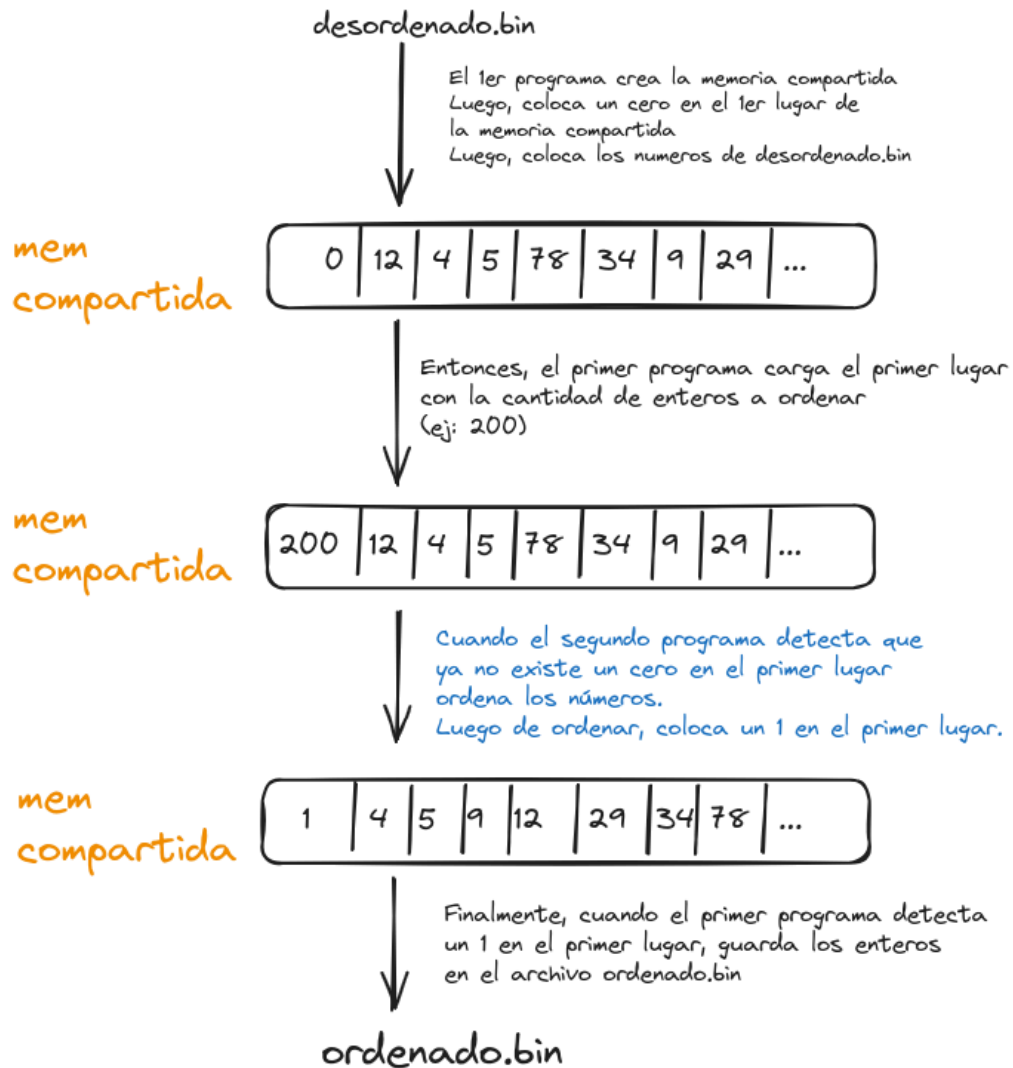
```
shm_fd= shm_open("PARCIAL", O_RDWR, 0666);
```

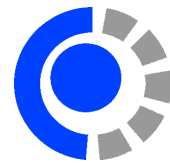
```
int * ptr= mmap(0, [aquí va el tamaño], PROT_READ|PROT_WRITE, MAP_SHARED, shm_fd, 0);
```

NOTA 2: Si se quiere utilizar la memoria compartida como si fuese un arreglo, se puede hacer así:

```
int * arreglo = (int *) ptr;  
arreglo[1] = 3; /* se modifica el segundo entero en la memoria compartida */
```

Descripción gráfica del ejercicio:





2. Desarrollar dos programas en **XINU** que sean productor consumidor **de números enteros**, reutilizando el buffer circular de 10 entradas desarrollado para el driver.

Para reutilizar el buffer, agregue 3 funciones:

`iniciar_buffer();`

`leer_dato();` : devuelve el próximo entero que tenga el buffer circular (sino bloquea).

`colocar_dato(int n);` : coloca el entero n en el buffer.

Además, deberá declarar el buffer como de enteros (int), y no de char.

En `colocar_dato()`, si el buffer está lleno, la función esperará a que haya un lugar (simplemente lea continuamente el valor del semáforo que controla el buffer, hasta que haya un lugar disponible). **NO UTILICE UN NUEVO SEMAFORO EN LO POSIBLE.**

Luego, desarrolle los dos programas productor consumidor:

El primer programa cargará los 100 primeros números naturales en el buffer. El segundo programa tomará valores desde el buffer y los muestra por la consola.

Si los 100 números naturales aparecen en la consola entonces significa que los dos programas trabajan correctamente sincronizados por el buffer circular.

3. Responda:

- Describa el sistema de pasajes de mensajes de XINU, con sus características (si es síncrono o asíncrono, si usa buffer, si no utiliza, etc).
- Defienda o critique la frase: Debido a que las aplicaciones utilizan el paradigma open-read-write-close para acceder a los dispositivos, tanto en XINU como en UNIX (Linux, MAC OS, etc), se hace difícil utilizar los dispositivos, porque se confunden con archivos.
- Defienda o critique la frase: Un proceso fue programado con threads. Si un thread realiza un `read()`, y el `read()` bloquea, todos los threads y el proceso se bloquea.
- Los desarrolladores de firefox y chrome decidieron que cada "pestaña" sea un proceso, en vez de un thread. ¿Por qué piensa que se tomó esta decisión?.